AmanAI Lab

amanailab.com · YouTube @AmanAILab · LinkedIn: aman-chauhan71

AI System Design Interview Questions

10 System Design Questions Commonly Discussed in AI Engineering Interviews

01	Design an LLM system answering a live user query
02	Design ChatGPT end-to-end (front-end to backend)
03	Reduce LLM token cost at scale
04	Design an AI coding assistant inside an IDE
05	Design RAG for a 10-million-document knowledge base
06	Design a chatbot on top of a third-party LLM
07	Design safeguards for an autonomous AI agent
08	Design a recommendation system for 10M users, zero downtime
09	Design a fraud-detection system at sub-100ms latency
10	Why GPU limits change your architecture vs. a normal backend

■ Free PDF Series: github.com/amanailab/Production-level-Generative-AI-Interview-Questions

00 How to Use This Guide

60 seconds — read this before Q1

Every answer below follows the same structure real interviewers grade you on: **Clarify** → **Architecture** → **Deep Dive** → **Numbers**. Read the **blue box** for what the interviewer asks next, the **red box** for the mistake most candidates make, and the **gold box** for the one line that makes you sound senior.

01 Design a high-level system for an LLM responding to a live user query.

Core ■ Commonly discussed in AI system design interviews

THE SETUP

This is the single most commonly reported opening prompt across every major AI company. It sounds simple, which is exactly the trap — the interviewer is watching whether you scope it before drawing a single box.

STEP 1 — ASK BEFORE YOU DESIGN

- Is this a general chat product, or does it need to answer from private/company data? (This single answer decides whether RAG is even in scope.)
- What latency do users expect — should the reply start streaming in under a second, or is a few seconds fine?
- Single question-answer, or a full multi-turn conversation with memory?
- Roughly what traffic — is this 100 queries/day or 100,000 queries/second? This one number changes almost every downstream decision.

STEP 2 — DRAW THE SYSTEM (explain out loud as you go)

1. User sends a request to an **API Gateway** — this handles authentication and basic rate limiting so a single user cannot flood the system.
2. The request hits a **Semantic Cache** check first — if a very similar question was asked recently, the cached answer returns instantly at near-zero cost.
3. If no cache hit, a **Model Router** looks at the query and decides: is this simple enough for a small, fast, cheap model, or does it need the large, expensive model?
4. The chosen model runs inside a **batched GPU serving layer** — many users' requests are grouped together so the expensive GPU is never sitting idle.
5. The answer is **streamed back token by token** over a live connection, so the user sees words appearing immediately instead of waiting for the full response.
6. Every request is **logged asynchronously** in the background — logging never slows down the response the user actually sees.

STEP 3 — WHAT THE INTERVIEWER PUSHES ON NEXT

- **"What if the model provider goes down?"** — Answer: have a second backup model ready, and a circuit breaker that automatically switches over so users never see a hard failure.
- **"How do you keep this affordable?"** — Answer: the model router is your main cost lever — sending simple questions to a small model instead of the large one can cut cost by well over half.

■ **Interviewer follow-up:** What changes in your design if traffic suddenly grows 10x or 100x overnight?

■ **Common mistake:** Jumping straight into drawing boxes without asking a single clarifying question — this is the #1 reason candidates lose points in the first two minutes.

■ **Pro tip:** Say the plan out loud before you draw anything: 'Let me first understand the scope, then I'll sketch the high-level flow, then we can go deep wherever you'd like.' This single sentence signals structure immediately.

02 Design ChatGPT — from the front-end experience to the backend.

Expert

■ A full-stack style question, not just backend

WHY THIS QUESTION IS DIFFERENT

Most system design interviews only test backend architecture. OpenAI is different — they expect you to reason across the FULL stack in the same conversation: what the user sees on screen, the API contract, and the backend serving the model. Treat this as three connected mini-designs, not one.

LAYER 1 — WHAT THE USER SEES

- Text should stream in word by word as the model generates it — waiting several seconds for a blank screen feels broken to users.
- The user's own message appears on screen the instant they hit send, before the server even responds — this is called an "optimistic" update, and it makes the app feel instant.
- Users need a way to regenerate a bad answer, or edit an earlier message and branch the conversation from there.

LAYER 2 — THE API CONTRACT

- A single endpoint accepts the conversation so far and streams the reply back live, rather than making the client wait for one giant response.
- Every conversation gets an ID so the server knows which chat history to continue — this is what lets you close the tab and come back later.
- Requests are rate-limited per account so one heavy user cannot degrade the experience for everyone else.

LAYER 3 — THE BACKEND

- Requests are routed to a serving cluster running multiple model versions side by side — this lets OpenAI roll out a new model to a small % of users first.
- Conversation history is kept in a fast short-term store (for the current chat) plus a longer-term memory store (for things the user wants remembered across chats).
- Every message — both what the user asked and what the model is about to say — passes through a safety/moderation check before and after generation.

THE FOLLOW-UP EVERY CANDIDATE REPORTS GETTING

Practice this drill for every design in this guide, not just this one: "If traffic grows 10x, what breaks first? What about 100x?" At 10x, your GPU pool for inference typically saturates first. At 100x, a single data-center region can no longer serve everyone, and now you're distributing huge model files to new regions — a genuinely hard problem on its own.

■ **Interviewer follow-up:** How would you get a multi-hundred-gigabyte model file onto thousands of new machines quickly, over a network that has limited bandwidth?

■ **Common mistake:** Only describing the backend and skipping the front-end streaming experience entirely — for this specific company, that's treated as an incomplete answer, not just a missed nice-to-have.

■ **Pro tip:** OpenAI interviewers have no fixed question bank — memorizing this exact prompt won't help as much as practicing the FULL-STACK habit: always sketch front-end, API, and backend together for any AI product question.

03 How would you reduce token costs in an LLM-powered product at scale?

Core ■ A frequent theme in LLM product design rounds

GIVE THE FRAMEWORK FIRST, THEN THE DETAIL

Structure your entire answer around exactly three levers. Naming the framework out loud before diving in shows the interviewer you have a mental model, not just scattered facts.

LEVER 1 — SEND EACH QUERY TO THE RIGHT-SIZED MODEL

Not every question needs your most powerful (and most expensive) model. A query classifier looks at each incoming question and decides: is this simple enough for a small, fast, cheap model, or does it genuinely need the large model's reasoning power?

cheap	20-70x costlier	70% simple queries
Small model	Large model	Typical mix

LEVER 2 — SHRINK WHAT YOU SEND TO THE MODEL

- Keep system instructions as short as possible — every extra word in your prompt is charged on every single request, forever.
- If your system reuses the same instructions repeatedly, some providers let you "cache" that fixed prefix so you're not paying to re-process it every time.
- When you're pulling in reference documents (RAG), compress or trim them to only the relevant sentences before handing them to the model — don't paste in entire documents.

LEVER 3 — AVOID CALLING THE MODEL AT ALL WHEN POSSIBLE

Many products see the same or very similar questions repeatedly. A semantic cache checks if a new question closely matches one already answered recently, and if so, returns that answer instantly — for free.

PUTTING IT TOGETHER — A REALISTIC EXAMPLE

Imagine 70% of your traffic is simple questions routed to a cheap model, and on top of that roughly 35-40% of ALL queries are cache hits that cost nothing. Combined, these two levers alone can realistically cut your total token spend by more than half — this is the kind of concrete number interviewers want to hear, not a vague "we'd optimize costs."

■ **Interviewer follow-up:** How do you make sure the cheaper model isn't quietly giving worse answers just to save money?

■ **Common mistake:** Only mentioning caching and stopping there — cost questions are almost always scored on breadth (naming multiple independent levers), not depth on just one.

■ **Pro tip:** Always pair each lever with a rough number, even an estimate. 'Model routing typically saves 50-70% of inference cost' sounds far more credible than 'model routing helps reduce cost.'

04 Design an AI-powered code assistant that helps developers debug inside their IDE.

Advanced ■ Tests IDE / developer-tool system thinking

ASK THIS FIRST — IT CHANGES YOUR WHOLE DESIGN

- Does the assistant only need to see the current file, or the whole repository?
- Should its answers be grounded in internal docs, GitHub issues, or Stack Overflow — meaning, do we need retrieval (RAG) at all?
- Are we talking about tiny inline suggestions as the developer types, or a chat window for deeper debugging conversations? These are two very different products.

THE KEY INSIGHT — THIS IS ACTUALLY TWO SYSTEMS, NOT ONE

The single biggest signal of seniority in this question is recognizing that inline code completion and chat-style debugging have completely opposite requirements — and refusing to force one architecture to do both jobs.

SYSTEM A — INLINE COMPLETION (must feel instant)

- Needs to respond in well under half a second, or it feels laggy and developers turn it off.
- Uses a small, fast model — not the most powerful one, because speed matters more than depth here.
- The surrounding code context (imports, current function) is pre-loaded and reused across keystrokes rather than re-sent every time — this is what makes sub-second responses possible.

SYSTEM B — CHAT-STYLE DEBUGGING (can afford to think)

- A few seconds of response time is perfectly acceptable here, since the developer is reading and thinking, not typing continuously.
- Can use a larger, more capable model, and can pull in retrieved documents (RAG) from the repo, internal docs, or past issues.
- Can call tools — for example, actually running the failing test or reading the real stack trace — rather than just guessing from a code snippet.

IF RAG IS IN SCOPE — KEEPING THE INDEX FRESH

A codebase changes dozens of times a day through commits. Re-indexing the entire repository on every commit is far too slow. Instead, only re-process the specific files that actually changed — this keeps the search index current within seconds, not hours.

■ **Interviewer follow-up:** How do you keep the code index up to date without re-processing the entire repository on every single commit?

■ **Common mistake:** Designing one single pipeline that tries to serve both instant inline suggestions and deep chat debugging — this almost always fails the latency requirement for one of the two use cases.

■ **Pro tip:** State the two-system split explicitly and early: 'I'd actually split this into two paths with very different latency budgets.' This one sentence is often what separates a strong answer from an average one.

05 Design a RAG pipeline for an internal enterprise knowledge base with 10 million documents.

Core

The most common AI-specific system design category

THE HARD CONSTRAINT TO NAME UPFRONT

The bar candidates are actually held to here is retrieval in well under 50 milliseconds, searching across terabytes of stored document embeddings. Say this number back to the interviewer — it shows you understood the real constraint, not just "make search fast."

CLARIFY FIRST

- How often does the underlying data change — daily, hourly, or constantly? This decides whether you need real-time updates or a nightly batch job is fine.
- Do answers need strict citations back to source documents, or is a best-effort summary acceptable?

THE PIPELINE — INGESTION (happens once, or on update)

1. Each document is broken into smaller chunks (a few hundred words each) — sending an entire 50-page document to the model at once would be slow and expensive.

2. Each chunk is converted into a numerical "embedding" that captures its meaning, and stored alongside metadata like department, date, and access permissions.

3. Documents are split ("sharded") by department or domain — this way, a legal team's question only ever searches the legal shard, not all 10 million documents.

THE PIPELINE — QUERY TIME (happens on every user question)

1. The user's question is also converted into an embedding.

2. **Metadata filtering happens BEFORE the expensive search** — if the user only has legal-department access, we filter down to that shard first, cutting the search space by well over 90% before we even start comparing embeddings.

3. A **hybrid search** combines exact keyword matching with meaning-based (semantic) matching — this catches both "find the exact contract number" style queries and "find documents about termination policy" style queries.

4. The top 20 or so candidate chunks are re-scored by a more precise (but slower) reranking step, and only the best 3-5 are sent to the language model to generate the final answer.

HOW THIS ACTUALLY HITS SUB-50MS

- Filtering by metadata first is what makes the biggest difference — searching a small relevant slice is far faster than searching all 10 million documents every time.
- An approximate (rather than exact) search algorithm trades a tiny bit of accuracy for a large speed gain — at this scale, that tradeoff is standard practice, not a shortcut.
- Keeping the search index and the model that uses it in the same physical data center region avoids slow network round-trips.

■ **Interviewer follow-up:** How do you handle a situation where the same policy exists in three different document versions across the knowledge base?

■ **Common mistake:** Describing only pure semantic (vector) search and never mentioning hybrid search —, interviewers treat 'vector search only' as a sign of shallow, outdated knowledge.

■ **Pro tip:** Explicitly connect the constraint to your design choice: 'Given the sub-50ms requirement, I'd filter by metadata before running the expensive similarity search, since reducing the search space matters more than a marginally faster search algorithm.'

06 Design a customer-support chatbot integrated on top of a third-party LLM platform.

Advanced

■ Tests integration and safety thinking, not model-building

THE KEY REFRAME

This is NOT the same question as "design ChatGPT." Here, you do not control or build the model — a third party (like an external LLM API) does. Your job is everything AROUND the model: making sure it stays grounded, safe, and has a graceful fallback plan.

DRAW A CLEAR LINE — "THEIRS" VS. "MINE"

- **Theirs:** the raw language model that generates text.
- **Mine:** grounding the model in our actual support content, filtering what goes in and out, and deciding when a human needs to step in.

GROUNDING — WHY YOU STILL NEED RAG HERE

A general-purpose third-party model has never seen your company's specific refund policy or product documentation. Retrieving your actual support articles and feeding them into the prompt before generating a reply is what prevents the bot from confidently making up a wrong policy.

SAFETY LAYERS AROUND THE THIRD-PARTY CALL

- **Before sending to the provider:** strip out any sensitive personal information from the user's message that doesn't need to leave your systems.
- **After receiving the reply:** run it through a content-safety check before the customer ever sees it.
- **Escalation:** if the model's confidence is low, or it detects a frustrated customer (repeated same question, negative tone), hand off to a human agent — with the full conversation already attached, so the customer never has to repeat themselves.

WHAT HAPPENS WHEN THE THIRD PARTY FAILS

Since you don't control the model provider, you must plan for their outages. Keep a second backup provider ready to switch to automatically — the chatbot should never simply go silent because one vendor is down.

■ **Interviewer follow-up:** If the third-party provider silently changes how their model behaves overnight, how would you even find out?

■ **Common mistake:** Designing this exactly like a from-scratch LLM product and forgetting that the core constraint here is integration and safety around a model you don't control.

■ **Pro tip:** Open your answer by naming the ownership boundary explicitly: 'I don't own the model here, so my design is really about grounding, safety, and fallback around it.' This framing alone signals strong systems thinking.

07 Design safeguards for an AI system that can take actions on behalf of a user.

Core

■ An increasingly common prompt as agentic products ship

WHY THIS QUESTION IS NOW STANDARD

As AI agents move from demos into real products that can actually click buttons, send emails, or spend money, this has become one of the most commonly asked prompts — because it tests judgment about risk, not just architecture skill.

STEP 1 — SORT EVERY POSSIBLE ACTION INTO A RISK TIER

- **Safe / read-only:** looking something up, checking a status — the agent can do these completely on its own.
- **Reversible write:** creating a draft, adding an item to a list — low risk, can be undone easily.
- **Irreversible:** sending a payment, deleting something permanently, sending a message to someone outside the company — these require a human to explicitly approve before they happen.

STEP 2 — HARD LIMITS THAT CANNOT BE TALKED AROUND

- A maximum number of actions allowed per task, so a confused agent can't loop forever.
- A spending cap per session, so a mistake cannot become an expensive one.
- A time limit on each individual step, so the agent can't get stuck waiting indefinitely.

STEP 3 — LIMIT WHAT THE AGENT IS EVEN ABLE TO TOUCH

The agent should only have access to exactly what a specific task requires — never broad, account-wide permissions by default. If it doesn't need to touch billing information to answer a support question, it simply shouldn't have access to billing information at all.

STEP 4 — MAKE EVERY DECISION REVIEWABLE AFTER THE FACT

Every single action the agent takes should be logged along with the reasoning that led to it. If something goes wrong, this log is what lets a human understand exactly what happened and, where possible, undo it.

■ **Interviewer follow-up:** How would you design the human-approval screen so people actually read it, instead of reflexively clicking 'approve' every time?

■ **Common mistake:** Describing only one blanket safety rule ('always ask a human first') instead of the tiered approach — interviewers specifically listen for you distinguishing between low-risk and high-risk actions.

■ **Pro tip:** There is no fully 'safe' answer here that skips human oversight entirely — the interviewer is testing whether you default toward caution on irreversible actions without being told to. Say the risk tiers out loud before anything else.

08 Design a recommendation system serving 10 million users, with model updates and zero downtime.

Advanced ■ Classic two-stage recommendation system design

THE ARCHITECTURE EVERY STRONG ANSWER USES

The single most common mistake here is describing one giant model that scores a user's entire catalog on every single request — that simply doesn't scale. The standard, expected pattern is a **two-stage design**.

STAGE 1 — CANDIDATE GENERATION (cheap, casts a wide net)

A fast, lightweight process narrows millions of possible items down to a few hundred reasonable candidates for this specific user — using simple similarity or "people who liked this also liked that" style logic. Speed matters far more than precision at this stage.

STAGE 2 — RANKING (expensive, but only on a few hundred items)

Now that the pool is small, a much richer, more expensive model carefully scores just those few hundred candidates using detailed signals — this is where the heavy computation happens, but only ever on a tiny slice of the catalog, never all 10 million users' worth of items at once.

UPDATING THE MODEL WITHOUT ANY DOWNTIME

1. **Shadow mode:** the new model runs silently alongside the old one, scoring the same requests, but its results are never shown to real users yet — this validates it's working correctly with zero risk.
2. **Canary rollout:** once shadow mode looks healthy, a small slice of real traffic — say 1-5% — is switched to the new model, and engagement is compared against the old one.
3. **Gradual ramp-up:** if metrics hold steady or improve, traffic is increased step by step until the new model fully replaces the old one — with the ability to instantly roll back at any point.

A TRAP THAT CAUSES REAL PRODUCTION BUGS

The features used to train a model must be computed the exact same way when the model is actually serving live recommendations. If training and serving compute a feature even slightly differently, the model behaves unpredictably in production despite testing perfectly offline — this is called training-serving skew, and it's one of the most common real-world ML failures.

■ **Interviewer follow-up:** How would you notice that a newly deployed model is quietly hurting engagement before real damage is done?

■ **Common mistake:** Proposing one single model that scores the entire catalog for every user on every request — always split into candidate generation (broad, cheap) then ranking (narrow, precise).

■ **Pro tip:** Mention canary rollout by name and describe the percentage ramp — '1-5% first, then gradually increase' is a concrete detail that signals real production experience, not just theory.

09 Design a fraud-detection system that must decide within sub-100ms at high transaction volume.

Core ■ Combines strict latency with continuous model freshness

WHY THIS IS A FAVORITE INTERVIEW SCENARIO

It combines two hard constraints at once: a very tight time limit to make a decision, AND a model that must keep adapting because fraud patterns change constantly. Very few system design questions test both simultaneously — that combination is exactly why it's asked so often.

HANDLING THE INCOMING FLOOD OF TRANSACTIONS

- Transactions stream in continuously through an event pipeline (rather than being processed one at a time from a queue with delays).
- As each transaction arrives, relevant signals are calculated in real time — for example, how many transactions this card has made in the last minute, or whether this device has been seen before.

MAKING THE DECISION WITHIN THE TIME LIMIT

- A smaller, faster model is used specifically because it needs to respond within milliseconds — the biggest, most accurate model is not always the right choice when speed itself is a hard requirement.
- Wherever possible, the features needed for the decision are pre-calculated ahead of time, rather than computed fresh during the transaction itself — computing things live in the critical path is what blows the time budget.

THE FALLBACK PLAN (interviewers specifically check for this)

If the machine learning service itself becomes slow or unavailable, the system should NOT simply block every transaction. Instead, it falls back to a simpler set of manual rules — this way, the business keeps running even during an ML outage, just with slightly less sophisticated fraud checks temporarily.

A SMART TWO-SPEED PATTERN

Many real systems run a fast, simple model in the critical real-time path to make the immediate allow/block decision, while a slower, more thorough model reviews the same transaction afterward and flags anything suspicious for a human to review later — combining speed where it's needed with accuracy where there's time for it.

WHY THE MODEL NEEDS CONTINUOUS RETRAINING

Fraudsters actively change their tactics once they realize a pattern is being detected. A feedback loop where confirmed fraud cases (and confirmed false alarms) flow back into retraining — ideally within days, not months — keeps the model from going stale.

■ **Interviewer follow-up:** How do you balance the cost of blocking a real, legitimate customer's transaction by mistake against the cost of letting an actual fraud through?

■ **Common mistake:** Forgetting to mention what happens when the ML service itself goes down — interviewers explicitly listed the fallback mechanism as something they check for in this exact question.

■ **Pro tip:** Say the fallback out loud even if not asked: 'If the ML service is unavailable, I'd fall back to a rules-based check rather than blocking everything' — this single sentence covers a specific, commonly graded point.

10 How do GPU memory limits and inference cost change your architecture compared to a normal backend system?

Advanced ■ Tests understanding of AI-specific infra constraints

THE ONE SENTENCE TO SAY FIRST

In a normal backend system, you handle more traffic by simply adding more cheap, identical servers. In AI serving, the hardware itself (GPUs) is expensive and has hard memory limits — so the architecture has to actively work around those limits, not just add more machines.

CONSEQUENCE 1 — YOU MUST BATCH REQUESTS TOGETHER

Running one user's request through the GPU at a time wastes a huge amount of its capacity — GPUs are built for parallel work. Grouping many requests together and processing them as a batch keeps the expensive hardware actually busy instead of sitting idle.

CONSEQUENCE 2 — A SINGLE MODEL MIGHT NOT FIT ON ONE GPU

Very large models can be too big to fit in one GPU's memory at all. The solution is splitting the model itself across multiple GPUs that constantly communicate with each other during every single request — this adds real complexity and overhead that a normal backend service never has to deal with.

CONSEQUENCE 3 — YOU CAN TRADE A LITTLE ACCURACY FOR A LOT OF SAVINGS

Reducing the numerical precision the model calculates with (a technique called quantization) shrinks both memory usage and cost significantly, in exchange for a small, usually acceptable, drop in accuracy — this is a tradeoff a normal web backend never has to consider.

CONSEQUENCE 4 — SCALING UP IS SLOW, NOT INSTANT

A normal web server can be spun up in a couple of seconds to handle a traffic spike. Loading a large AI model onto a fresh GPU can take tens of seconds by comparison — meaning a sudden burst of traffic can't be absorbed nearly as quickly as it can in a typical backend system.

■ **Interviewer follow-up:** If you suddenly need 3x the GPU capacity within the next 60 seconds, but loading a model takes tens of seconds, how would you handle that spike?

■ **Common mistake:** Answering this question with only AI vocabulary (mentioning 'GPUs' and 'quantization') without explaining WHY each one forces a different architectural decision — interviewers are testing understanding, not buzzword recall.

■ **Pro tip:** Ground every point in a concrete consequence, not just a term. Instead of 'we'd use quantization,' say 'we'd use quantization because it directly reduces memory pressure, which is the actual bottleneck here.'

00 The Night-Before Cheat Sheet

All 10 answers compressed to one line each

LLM answering a live query?	Scope first (grounded? latency? scale?) → gateway → cache → model router → batched serving → stream.
Design ChatGPT?	Full stack: streaming UI + API contract + backend serving. Always run the 10x/100x breaking-point drill.
Reduce LLM token cost?	3 levers: route to the right-sized model + shrink what you send + cache repeated answers.
AI code assistant in IDE?	Two systems, not one: instant inline completion (no RAG) vs. thoughtful chat/debug (RAG + tools).
RAG for 10M documents?	Shard by domain, filter by metadata BEFORE search, hybrid search, rerank top-20 down to top-5.
Chatbot on third-party LLM?	You own grounding + safety + fallback, not the model. Always have a backup provider ready.
Safeguards for AI agent actions?	Risk-tier every action (read-only / reversible / irreversible). Hard limits. Full audit log.
Recsys for 10M users, zero downtime?	Candidate generation (cheap, broad) then ranking (expensive, narrow). Shadow mode, then canary rollout.
Fraud detection sub-100ms?	Streaming features, small fast model, always a rules-based fallback if the ML service is down.
GPU limits vs normal backend?	Batching is mandatory, models may need splitting across GPUs, quantization tradeoff, slow cold start.

The habit that separates a pass from a borderline result: for every answer above, volunteer the failure mode and a rough cost number BEFORE the interviewer asks. Then run the 10x/100x drill on your own design out loud. This exact pattern is reported as the single most predictable evaluation signal across AI system design loops.